# Бизнес-задача

## Постановка бизнес-проблемы

Для мобильного ассистента крайне важен кейс, когда пользователь узнаёт информацию о погоде через ассистента. Список сценариев может быть широким. Вот некоторые из них:

- узнать погоду сейчас в месте, где находится пользователь
- помочь спланировать поездку: если погода будет плохой, пользователь предпочтет поехать в другое место
- И так далее

Необходимо подготовить PoC решение, чтобы составить дорожную карту по дальнейшей интеграции навыка в ассистента

## Требования к PoC

Рамках PoC-решения необходимо подготовить агента, который:

- позволяет решить первые два обозначенных кейса в бизнес-проблеме
- потенциально возможно расширять на другие подобные сценарии
- реализован с помощью инструментов Yandex Cloud AI Studio
- способен интегрироваться с другими сервисами Yandex

## План действий

Требования понятны. Теперь опишем план, как мы будем достигать поставленную цель:

- Сначала создадим свой MCP-сервер поверх API Я.Погоды
- Подключим его к агенту и проверим работоспособность
- Почему одного MCP для API Я.Погоды недостаточно? Как узнавать погоду не только по координатам, но и по названию локации?
- Учимся определять координаты по названию локации – делаем MCP-сервер с помощью API Я.Карт и Yandex Cloud Functions
- Подключаем к агенту с погодой и проверяем, что теперь можно писать адрес текстом, а не передавать координаты

# MCP-сервер для прогноза погоды

## Введение

Чтобы агент мог получить информацию о погоде, нам необходимо использовать MCP-сервер, который будет обращаться в API Яндекс.Погоды, получать результат и отправлять его в агента.

Немного отвлечёмся. В процессе развития агентной системы могут возникать ситуации, когда продукт уже реализовал MCP-сервер, но не готов оперативно выдать к нему доступ. При этом есть публичное API для работы с этим продуктом. Или же бывают ситуации, когда продукт вообще не имеет MCP-сервер, но у него также есть API. Могут возникать и другие ситуации, но в любом случае их объединяет одно – есть бизнес потребность быстро проработать сценарий в формате Proof-of-concept (PoC), чтобы понять стоит ли дальше вкладываться в него или имеет смысл направить ресурсы на другие бизнес-кейсы.

В таких случаях полезно уметь быстро прототипировать собственный MCP-сервер, с которым можно гибко экспериментировать для проработки сценариев ассистента. Такая ситуация возникла и сейчас, поэтому давайте реализуем подобный MCP-сервер самостоятельно, чтобы вы могли начать прорабатывать кейс с погодой в ассистенте уже в ближайшее время,.

В данном примере мы реализуем 2 кейса: получение информации о текущей погоде и предсказание погоды на ближайшие 2 дня.

## Реализация и запуск

Мы подготовили исходный код MCP-сервера, чтобы вы могли быстрее разобраться с базовыми техническими деталями. Исходный код находится в архиве weather-mcp.zip. Данная реализация написана на языке Python 3 и использует фреймворк fastmcp, который позволяет удобно переиспользовать общие компоненты и концентрироваться только на логике MCP-инструментов без необходимости писать много бойлерплейт кода.

Код из архива необходимо перенести на виртуальную машину и запустить его. Для облегчения этого процесса мы подготовили Dockerfile, который упрощает процесс деплоя приложения и требует выполнения всего несколько простых действий. Информацию о том, какие именно действия необходимо сделать, вы можете в файле README.md – он также расположен внутри архива weather-mcp.zip.

Для запуска вам необходима виртуальная машина, к которой есть доступ из интернета. Вы можете использовать собственный сервер или создать виртуальную машину в Yandex Cloud. Подробнее о том, как это сделать, вы можете прочитать в документации в разделе “Creating a Linux VM”.

### Тестирование с помощью MCP Inspector

Когда вы успешно развернули MCP-сервер на виртуальной машине, бывает полезно сначала протестировать его отдельно. Так вы сможете убедиться, что сам сервер отрабатывает корректно. Для этого можно использовать разные способы. Один из распространённых — использовать MCP Inspector.

Вы можете ознакомиться, как пользоваться MCP Inspector в официальной документации или в данной статье.

Важный момент. Чтобы проверить нашу реализацию MCP-сервера для прогноза погоды:

- Необходимо выбрать транспорт “Streamable-HTTP”
- Необходимо указать адрес до сервера в таком формате: “http://<ip_address>:<port>/mcp”, где ip_address – это IP-адрес вашей виртуальной машины, а port – это порт, который слушает MCP-сервер (например, 8000).

После этого можно нажать кнопку “Connect” и подключение должно стать активным.

Далее необходимо протестировать оба инструмента: получение информации о текущей погоде и предсказание погоды на ближайшие 2 дня.

Для этого:

- Выбираем вкладку “Tools”
- Нажимаем кнопку “List Tools”
- Выбираем инструмент (например, “current_weather”)

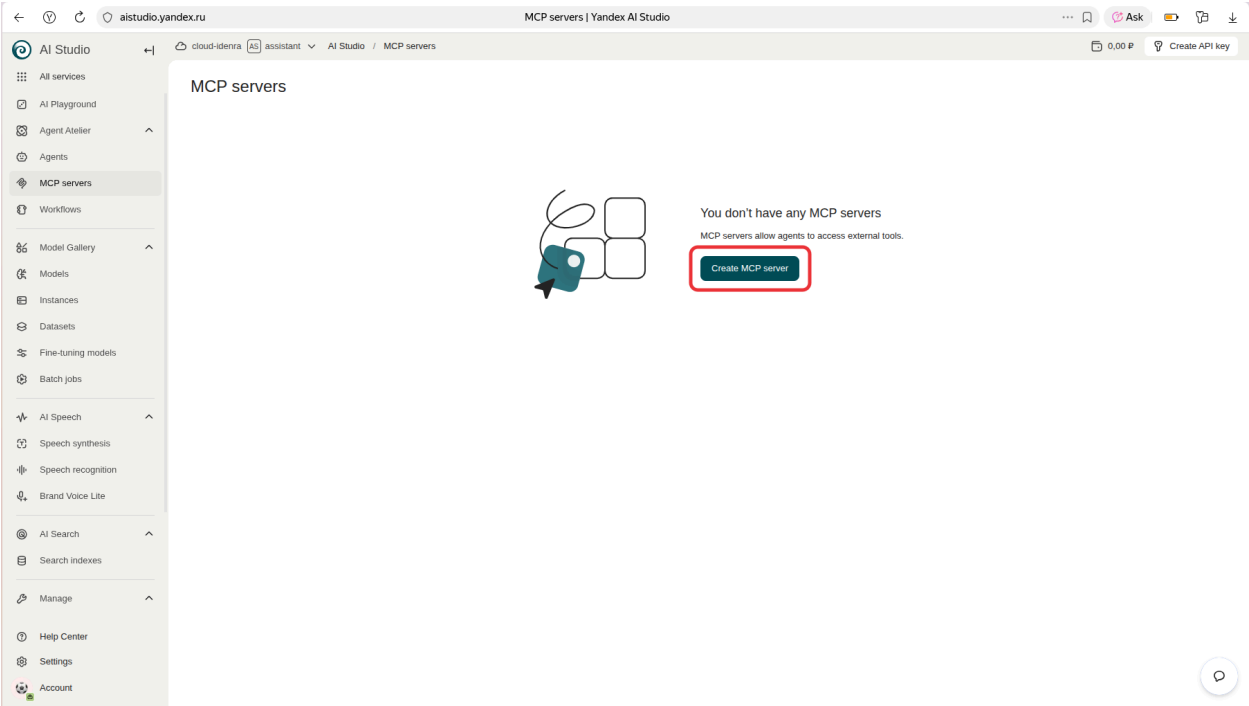
Справа появится окно с описанием инструмента и его параметрами (долготой и широтой). Заполняем параметры и нажимаем кнопку “Run Tool”.

Если всё сделано правильно, то вы увидите ответ от сервера.

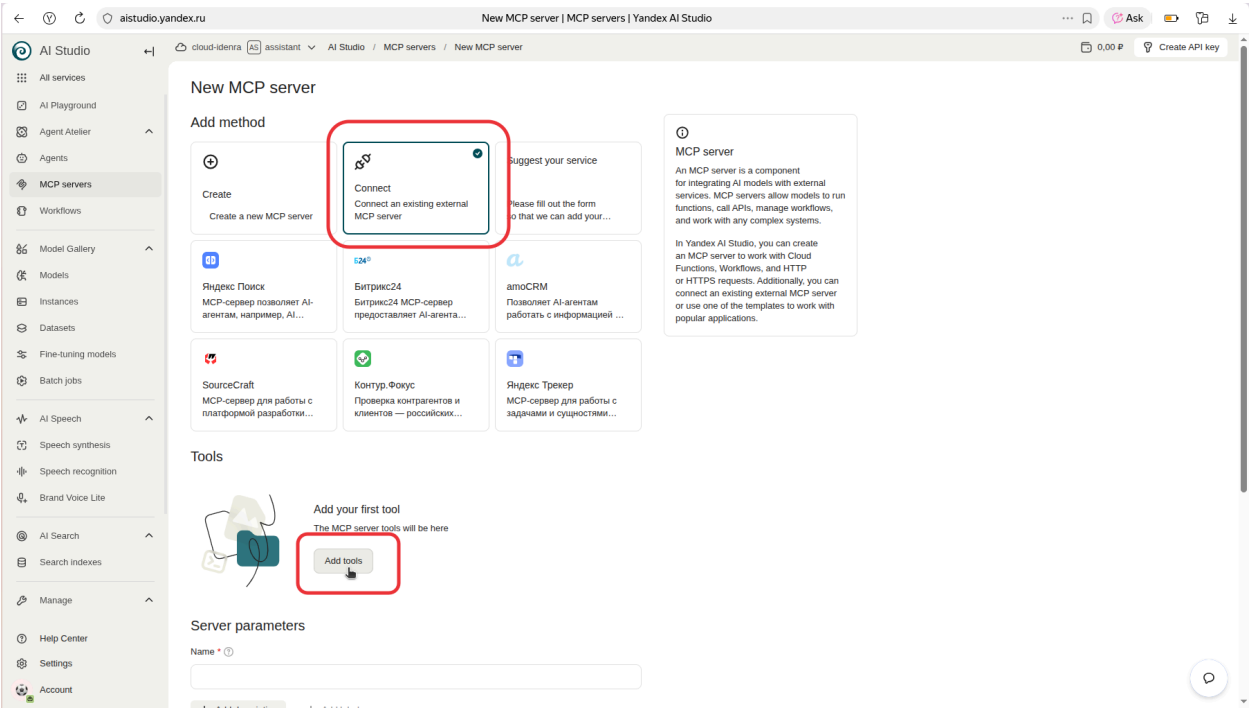
## Подключение MCP-сервера к агенту

### Регистрация в MCP Hub

Чтобы подключить ваш MCP в AI Studio, откройте раздел “MCP servers” и нажмите кнопку “Create MCP Server”



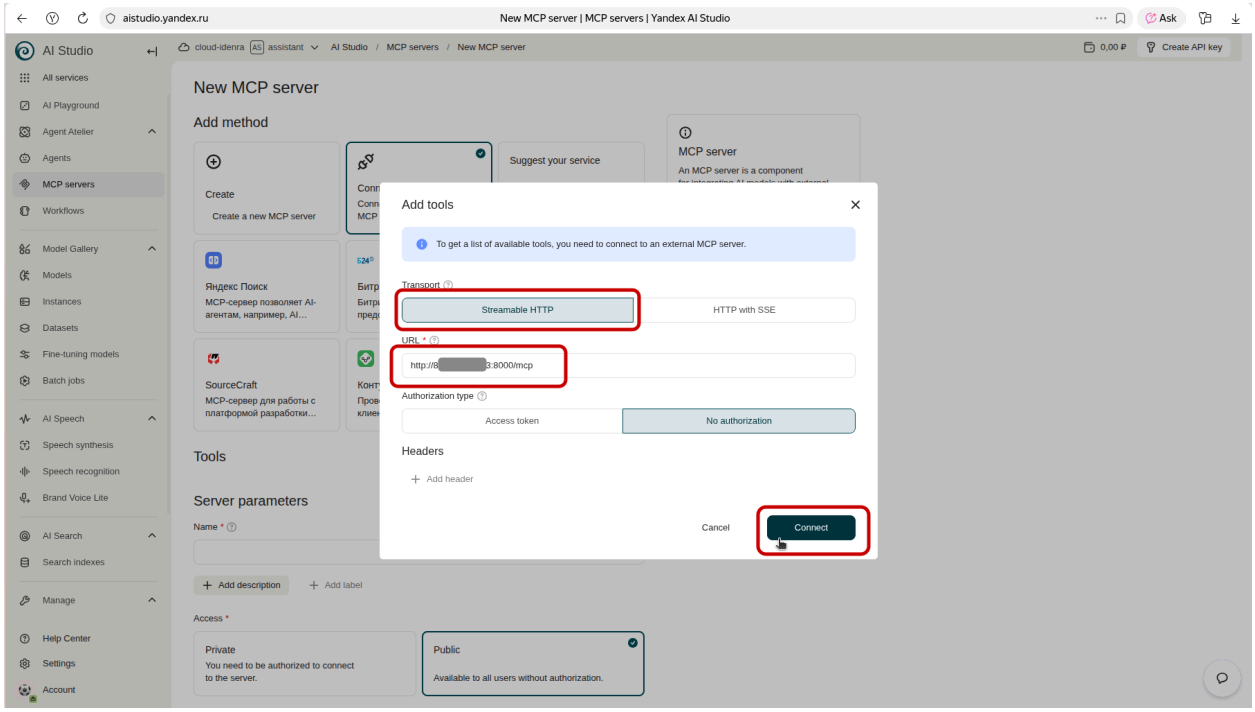

Далее выберете вариант “Connect” и затем нажмите кнопку “Add tool”



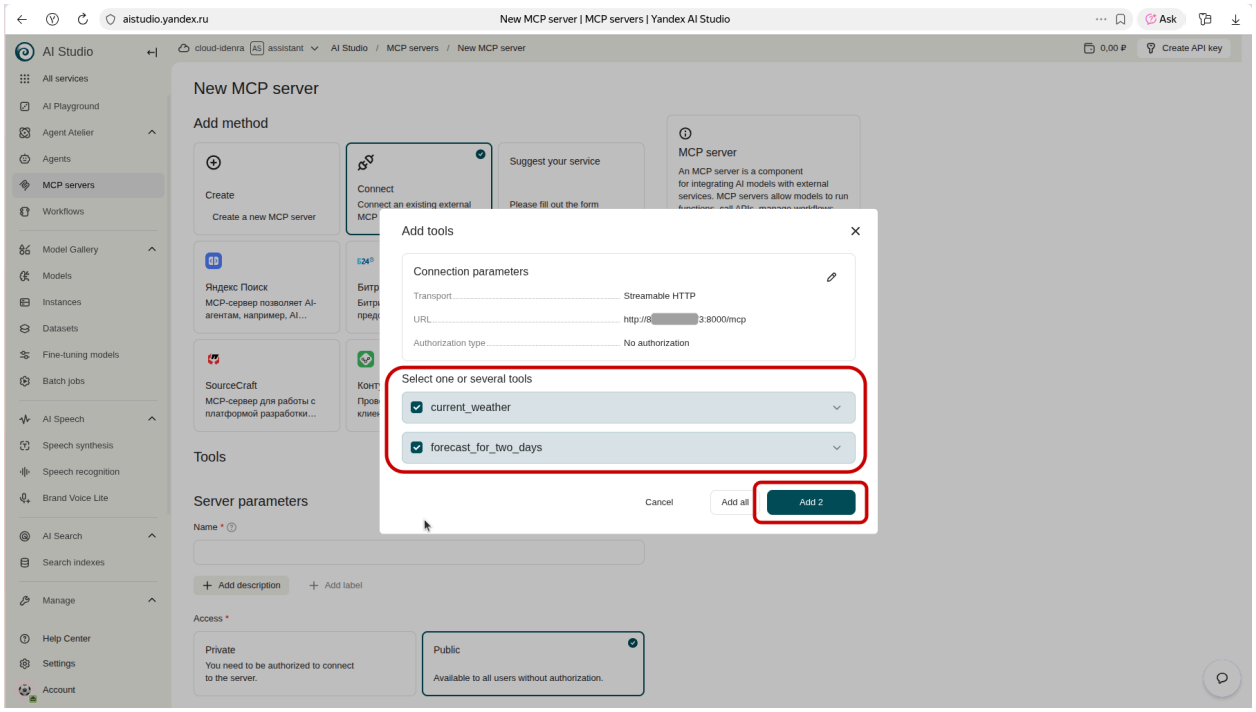

В открывшемся диалоговом окне вам необходимо:

- Выбрать транспорт “Streamable HTTP”

- Ввести URL MCP-сервера в формате “http://<ip_address>:<port>/mcp”, где ip_address – это IP-адрес вашей виртуальной машины, а port – это порт, который слушает MCP-сервер (например, 8000).
- Нажать кнопку “Connect”




Выберете все необходимые вам инструменты, которые предоставляет MCP-сервер. В нашем случае все инструменты. Далее нажмите кнопку “Add”.



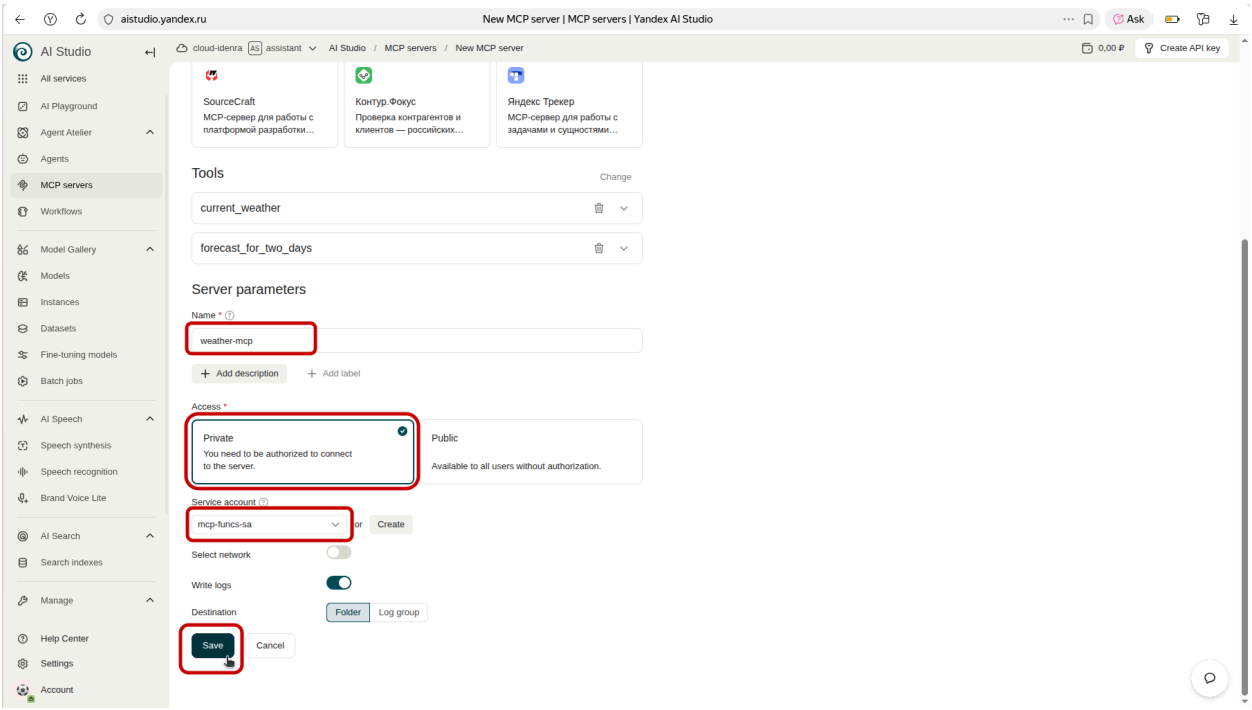

Теперь осталось указать настройки сервера в рамках MCP Hub.
В поле “name” введите название сервера. Используйте только латинский алфавит, цифры и дефис.

Блок “Access” позволяет настроить доступ к серверу в рамках AI Studio. Если выбрать “Public”, все пользователи AI Studio смогут обращаться к созданному серверу. Если выбрать “Private”, то при подключении к серверу (например, в агентах) AI Studio будет проверять, разрешено ли данному пользователю или сервисному аккаунту обращаться к нашему серверу. Давайте выберем вариант “Private”.

Настройка “Service account” нужна для того, чтобы MCP-сервер мог обращаться к другим сервисам Yandex Cloud. Для этого нужно создать отдельный сервисный аккаунт. Подробнее о том, как создать сервисный аккаунт, можно прочитать здесь, а инструкция, как назначить роли сервисному аккаунту, находится здесь.

В нашем случае необходим сервисный аккаунт с ролями “serverless.mcpGateways.invoker” и “functions.functionInvoker”, т.к. в следующем разделе мы будем создавать MCP-сервер на основе Yandex Cloud Functions. Как только вы создадите сервисный аккаунт, выберете его в блоке “Service account”.

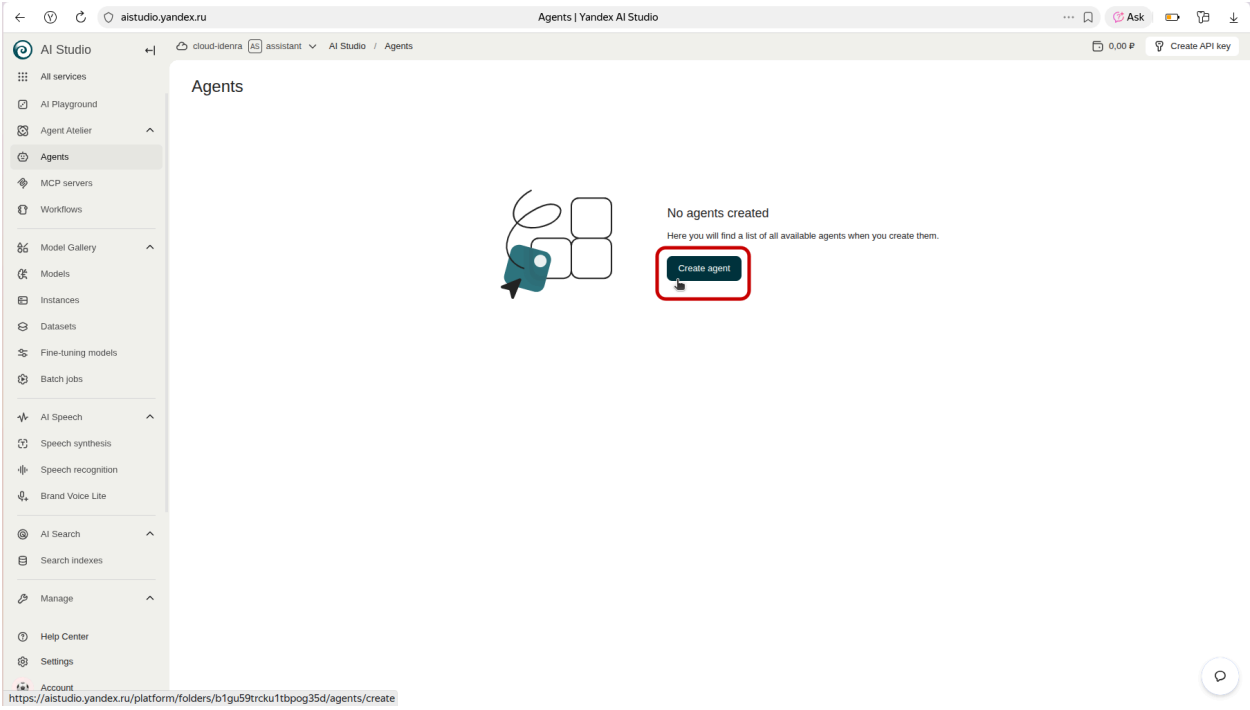
Сейчас у нас выставлены все необходимые настройки. Осталось нажать “Save”, чтобы наш сервер появился в MCP Hub.




## Подключение к агенту

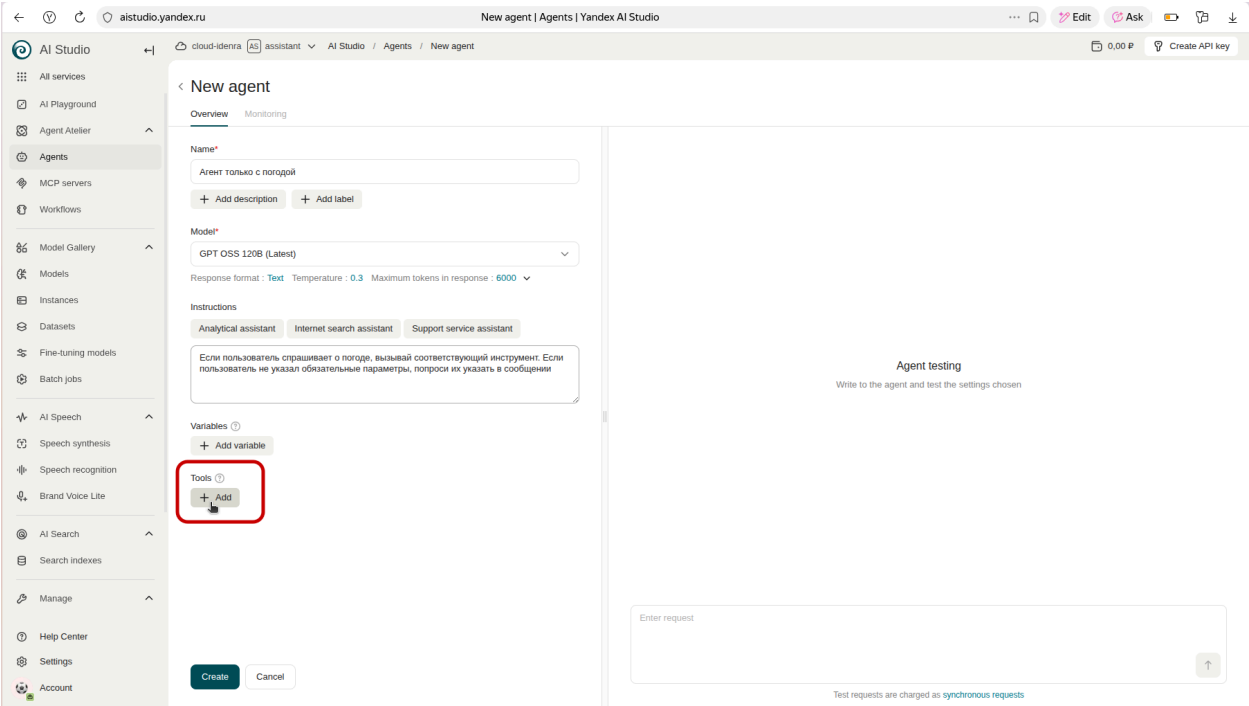
Давайте, наконец, подключим MCP-сервер с погодой к агенту и протестируем его.

Для этого перейдите во вкладку “Agents” и нажмите кнопку “Create agent”

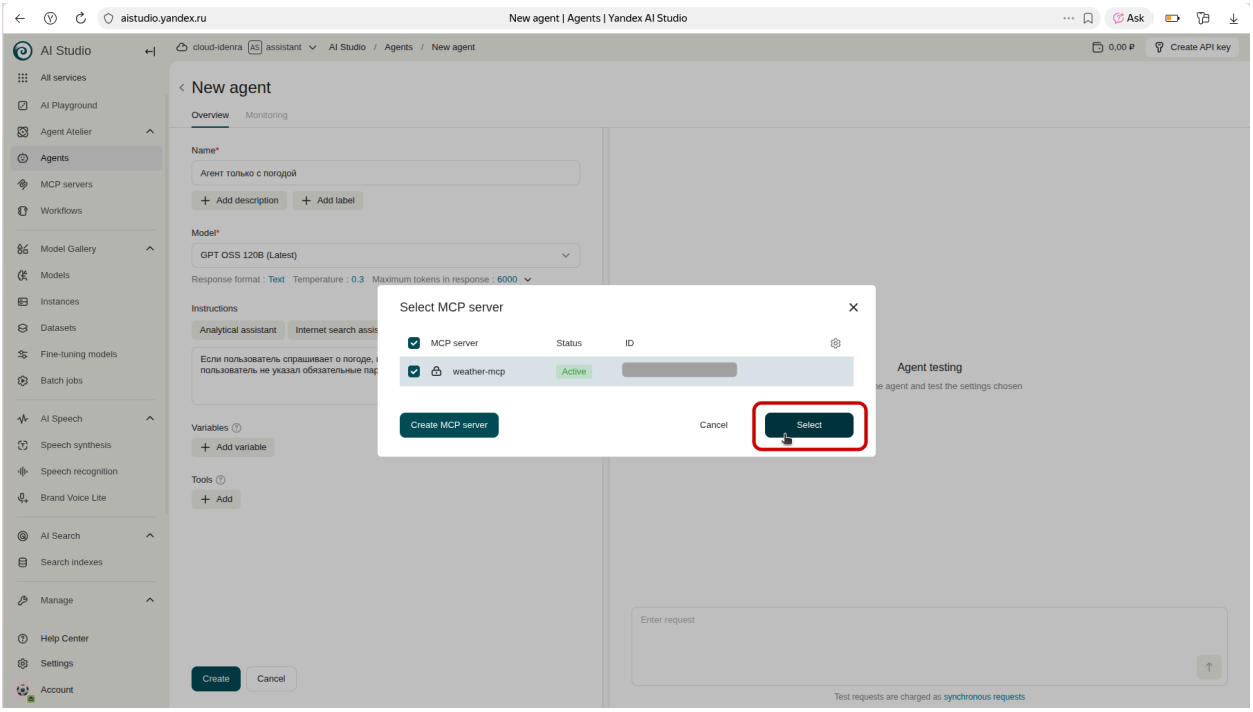



В открывшемся окне введите имя агента (поле “Name”), выберете модель с рассуждениями (например, gpt-oss-120b или DeepSeek 3.2) и введите инструкцию для агента. Вы можете написать свою инструкцию или воспользоваться заготовленной инструкцией “only-weather-agent.txt”.

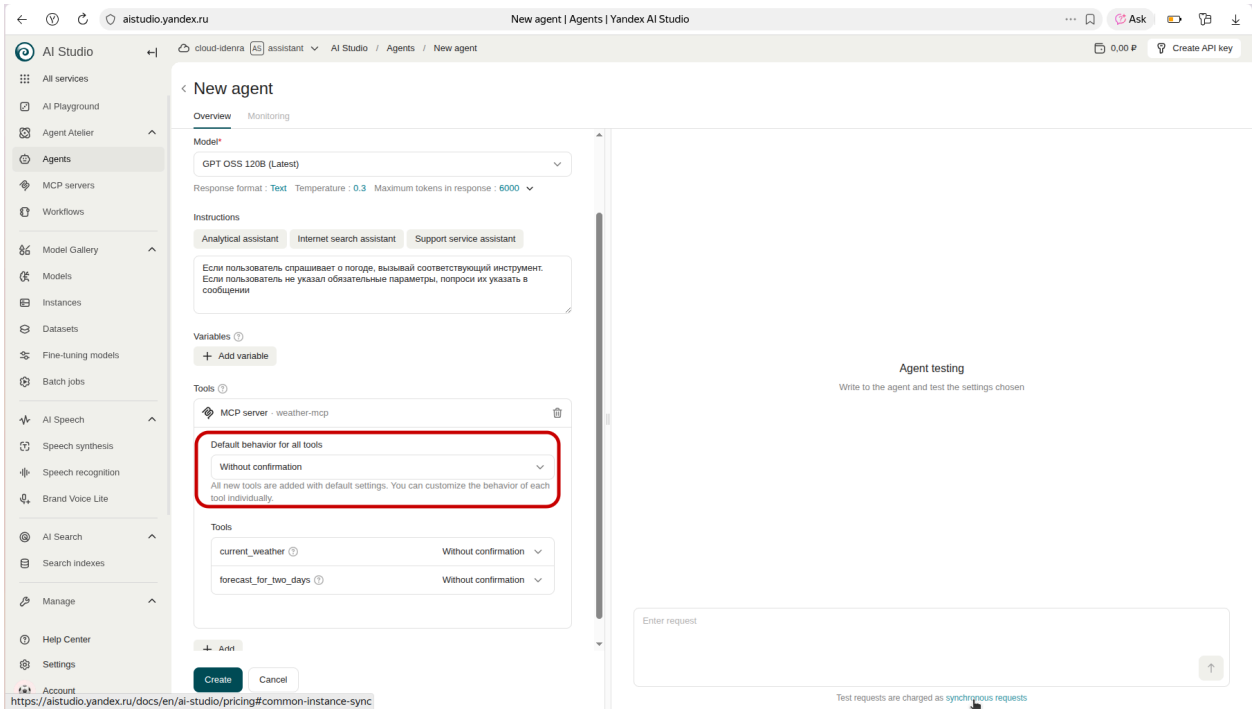
Теперь ключевое – подключение MCP-сервера. Найдите блок “Tools”, нажмите “Add” и выберете “MCP server”.




В открывшемся диалоговом окне выберете наш сервер и нажмите кнопку “Select”.




Остался последний штрих. Нужно решить, нужно ли агенту получать подтверждение от человека, когда агент захочет вызвать подключенный инструмент. Для нашего примера можем выбрать вариант “Without confirmation” для всех инструментов работы с погодой.




Осталось нажать кнопку “Create” и наш агент готов.

## Тестирование агента с погодой

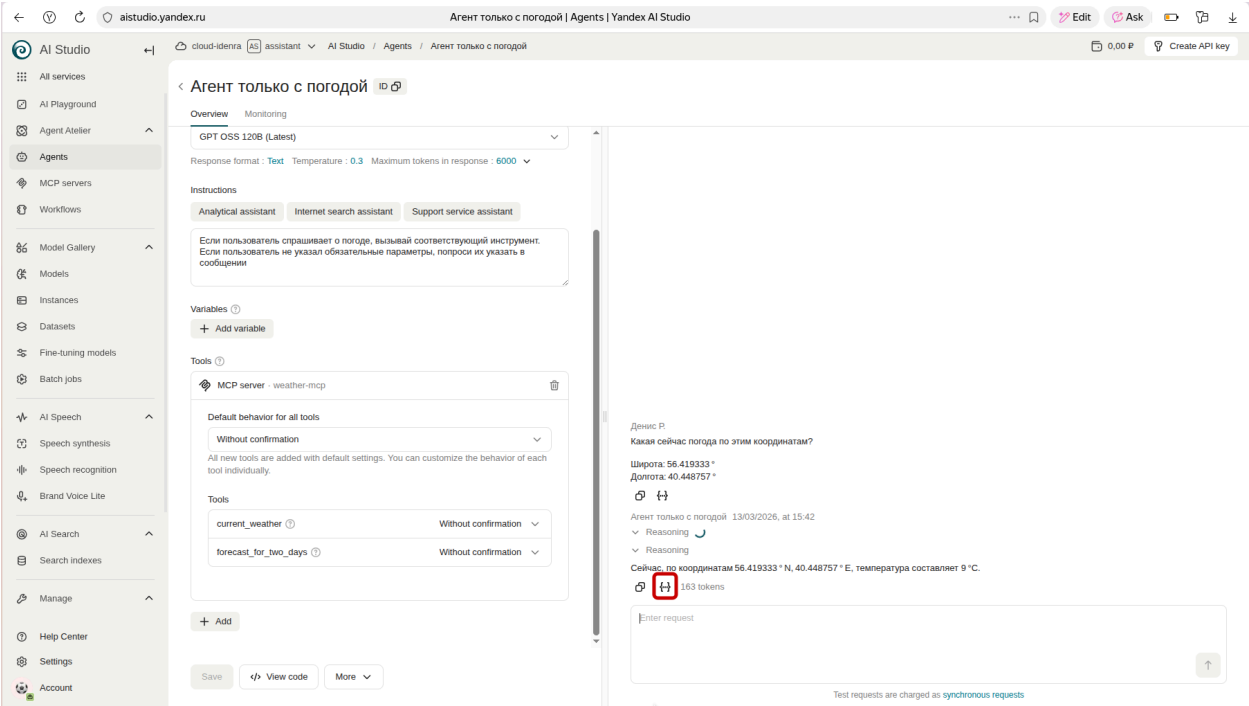
Давайте попробуем протестировать агента. Для этого зададим вопрос о текущей погоде, например, так:
“”

Какая сейчас погода по этим координатам?

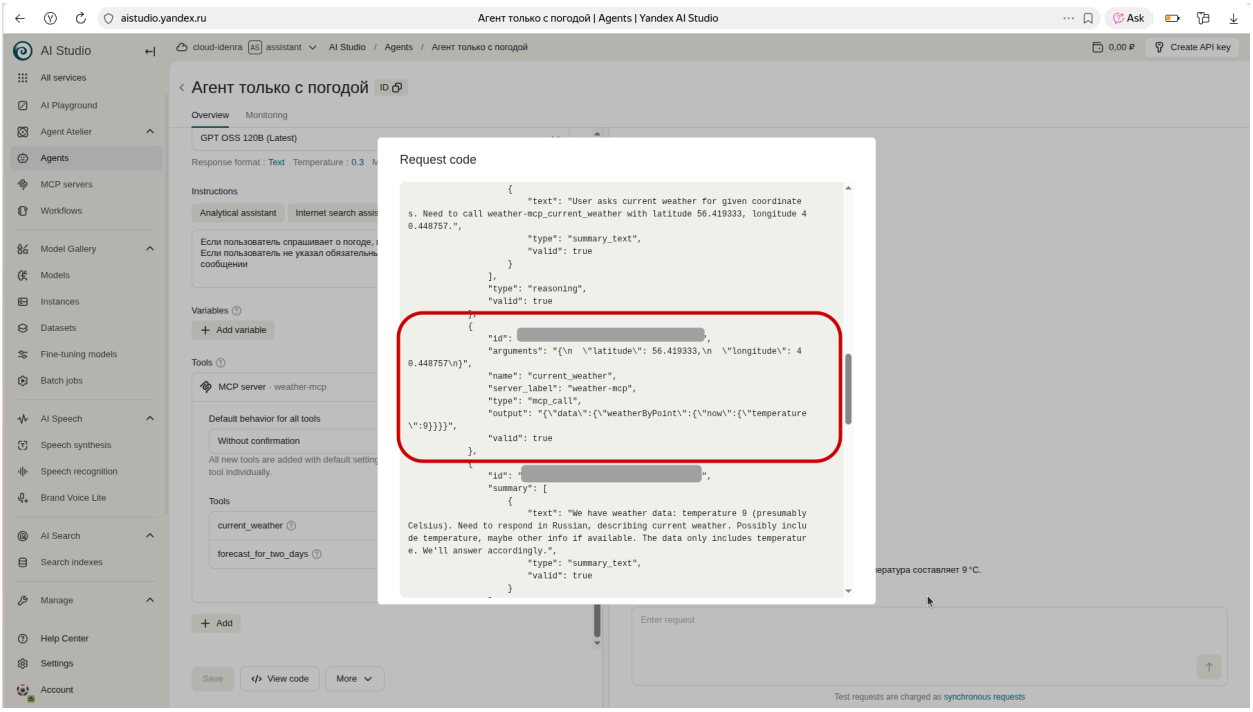
Широта: 56.419333 °
Долгота: 40.448757 °
“”

И мы получили верный ответ - ура!

Чтобы убедиться, что данные пришли из MCP-сервера, давайте посмотрим детали о работе агента на этом запросе. Для этого нажмите на кнопку “{...}” внизу под ответом агента.




Пролистаем вниз и увидим вызов (тип “mcp_call”). Выглядит он примерно так, как показано на изображении ниже:

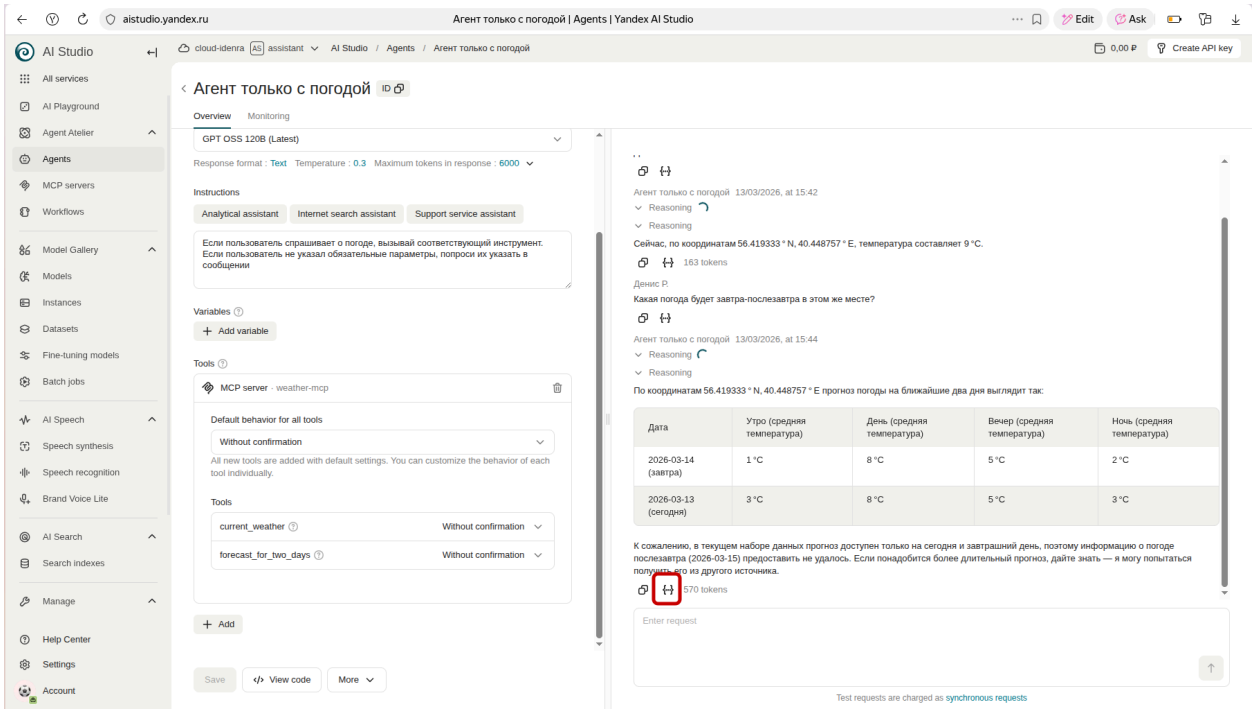



Теперь давайте проверим второй инструмент. Например, таким вопросом:
“”

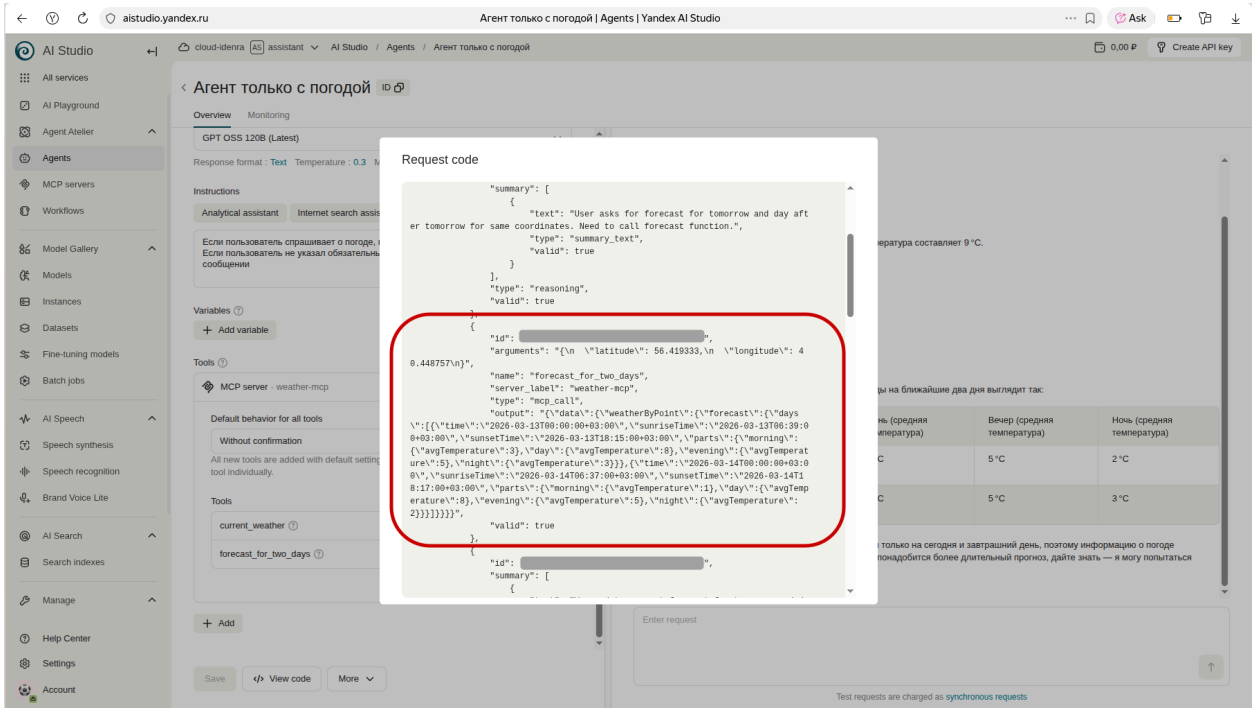
Какая погода будет завтра-послезавтра в этом же месте?

“”

Агент корректно отработал и на этом вопросе.




Убедимся в этом тем же способом, что и выше:



# MCP-сервер для Я.Карт

## Мотивация

Мы с вами реализовали и подключили к агенту собственный MCP-сервер для взаимодействия с Я.Погодой. Давайте подытожим, что мы имеем сейчас:

- Мы научили агента отвечать на вопросы о погоде;
- Мы настроили инфраструктуру, чтобы быстро проверять дополнительные сценарии, связанные с погодой. Например, может возникнуть идея собирать дайджест дождливых дней на ближайшие 2 недели. Для этого нам потребуется всего лишь написать один новый инструмент (1 файл), пересобрать сервер на виртуальной машине и подключить новый инструмент к агенту.

Однако есть и недостатки. Ключевой из них – пользователь должен явно передавать географические координаты в своем запросе. Такой сценарий в реальной жизни будет возникать крайне редко. Обычно люди обозначают локацию привычными названиями (город, страна и т.д.). Особенно это важно при голосовом взаимодействии с ассистентом.

Чтобы устранить этот ключевой недостаток, нам с вами необходимо добавить инструмент для гео-кодирования, т.е. превращения названия локации в её географические координаты.

Может быть несколько способов для решения этой задачи. Вот два наиболее очевидных решения:

- Использовать функциональность “Geosuggest” через API Я.Погоды
- Использовать Geocoder API от Я.Карт

Разберём оба варианта:

- Функциональность “Geosuggest” от Я.Погоды.
  Данный способ позволяет находить топонимы для поиска по запросу. Плюс данного способа в том, что для поиска координат мы можем использовать тот же самый API, который использовали для прогноза погоды. Минус в том, что будет использоваться та же квота на запросы, а также в API Я.Погоды работа с географией ограничена только данной фукциональностью
- Geocoder API от Я.Карт.
  Данный способ предоставляет значительно больше возможностей по работе с координатами – это плюс. Также для него используются другие квоты — это тоже плюс, т.к. потребление запросов о погоде и потребление запросов о географии разделяются – ситуация с потреблением более прозрачная для нас, а также мы можем более гибко управлять этим потоками данных. Минус состоит в том, что это

отдельное API, а значит способ взаимодействия отличается от того, как мы этого делали с Я.Погодой.

Предположим, что мы решили выбрать именно Geocoder API от Я.Карт, потому что они предоставляют больше возможностей для работы с гео-данными, которые могут понадобиться в обозримом будущем. Например, мы захотим добавить в нашего ассистента сценарий, когда пользователь попросит ассистента найти ближайшие от него рестораны, чтобы пообедать. В этом случае мы можем взять текущую геолокацию пользователя и найти ближайшие рестораны с помощью функции геокодера "Search for objects in a specified area".

Таким образом, с одной стороны, мы понимаем, что в будущем нам может понадобиться вся мощь Geocoder API. С другой стороны, сейчас мы решаем только одну задачу – научиться определять географические координаты по названию локации. Для нас важен быстрый результат, и мы не готовы тратить много времени и ресурсов для проверки одной небольшой гипотезы. В частности, мы не готовы реализовывать ещё один MCP-сервер таким же образом, как мы это делали с API Я.Погоды.

Для таких случаев бывает полезно использовать сервис бессерверных вычислений, который позволяет быстро реализовать бизнес-логику без дополнительных расходов по времени и деньгам. Примерами таких сервисов являются AWS Lambda, Alibaba Cloud Function Compute и так далее. Подобная возможность существует и в рамках Yandex Cloud. Сервис так и называется – Yandex Cloud Functions.

Удобно то, что в AI Studio можно реализовать MCP-сервер напрямую через Yandex Cloud Functions. Такая интеграция позволяет быстро проверить гипотезу и уже после принимать решение о дальнейших шагах.

Давайте попробуем реализовать MCP-сервер с помощью Yandex Cloud Functions

## Реализация и запуск

Мы подготовили исходный код MCP-инструмента для обращения к API Yandex Geocoder. Исходный код написан на Python 3 и находится в zip-архиве "geocoder-tool.zip".

В Yandex Cloud Functions можно загрузить весь исходный код прямо из zip-архиве Подробнее о том, как это сделать, можно прочитать в документации Yandex Cloud Functions.

Также, чтобы не хранить в коде API-ключ от Геокодера, давайте передадим его через переменные окружения. Для этого выполните следующие действия:

1. Найдите секцию "Environment variables" в редакторе функции
2. Введите название переменной окружения – GEO_API_KEY

3. Введите API-ключ в поле Value

Finally нажмите на кнопку “Save changes”. Теперь наша функция полностью готова к использованию в MCP Hub.

Важное уточнение. В данной функции мы описали только логику обращения к API. Однако нам нужно получать параметры от агента. Они будут приходить в объекте event. В исходном коде вы можете посмотреть детали, как именно это реализуется.

Также более безопасно передавать в функцию секреты через Yandex Cloud Lockbox. Подробнее, как создать секрет, вы можете посмотреть в документации, а в самой функции только ссылаться на них в секции “Yandex Lockbox secrets”.

### Тестирование

Данная имплементация содержит в себе только логику отдельного инструмента MCP-сервера, а не реализует весь интерфейс MCP-сервера, как это было в случае с API Я.Погоды.

Такой подход позволил нам значительно быстрее реализовать логику, но теперь мы не можем протестировать её, используя MCP Inspector – он работает только с полноценным MCP-сервером.

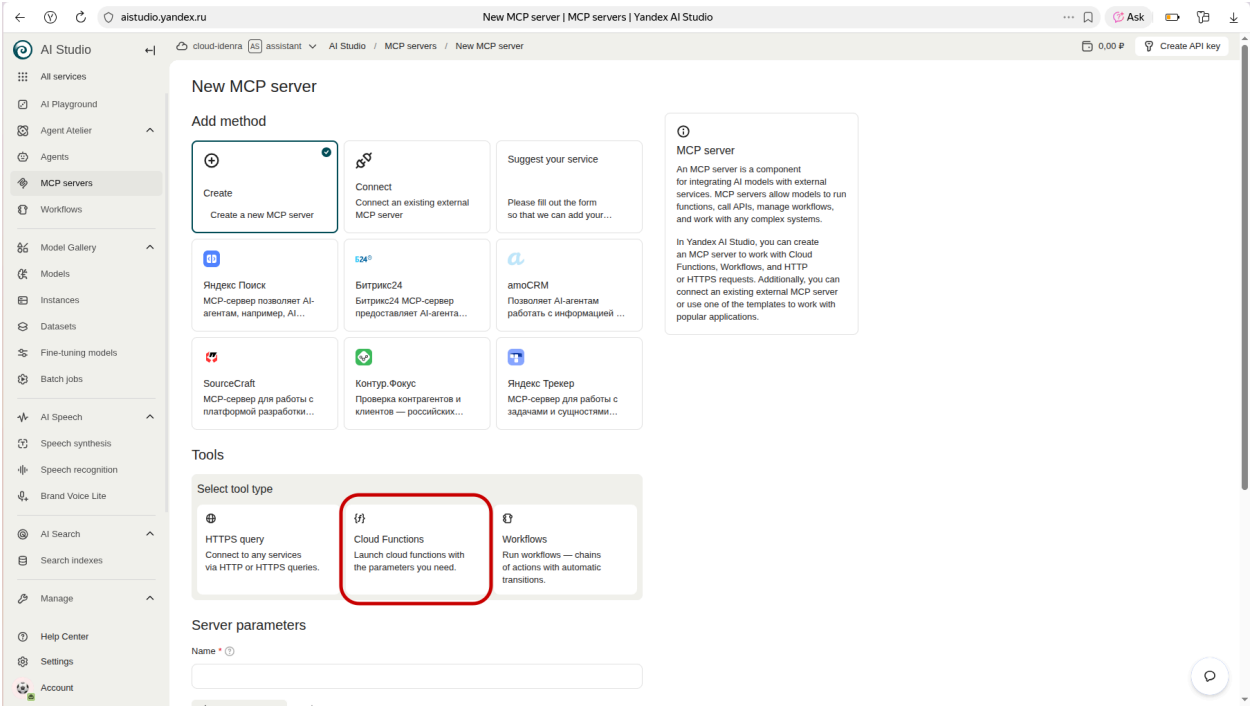
К счастью, нам это и не требуется. Мы можем протестировать разработанный инструмент напрямую через агента. Если нам потребуется отладить код в функции, мы сможем это сделать через встроенные логи в YC Functions. Для этого достаточно вывести отладочную информацию в stdout.

## Подключение к агенту

### Регистрация в MCP Hub

Регистрация инструмента из Cloud Functions в MCP Hub похожа на то, как мы это делали ранее с MCP-сервером Погоды.

Откройте раздел “MCP servers” и нажмите кнопку “Create MCP Server”. Появится экран как на изображении ниже. В блоке “Tools” необходимо выбрать вариант “Cloud Functions”




Появилось 2 подблока: описание инструмента и параметры инструмента.

В описании инструмента нужно указать имя инструмента, инструкцию для агента (именно на неё будет опираться агент при вызове функции) и саму Cloud Function, которую мы подготовили выше.

Наш пример инструкции для агента вы можете найти в приложенных материалах – “mcp-instructions/geocoder-instructions.txt”. Мы кратко описали функциональность инструмента, а также приложили формат ответа от Geocoder API, чтобы агент мог взять для своего ответа необходимые данные самостоятельно.

В подблоке параметры инструмента необходимо указать параметры, которые агент должен заполнить перед вызовом инструмента. В нашем случае только один параметр – address с типом String и описанием “Адрес или название организации”.

Осталось заполнить только блок “Server parameters”. Вы можете сделать это по аналогии с тем, как мы заполняли этот блок для MCP-сервера с погодой.

Осталось нажать кнопку “Save” и на этом MCP-сервер для геокодирования будет готов.

## Подключение к агенту

Данный MCP-сервер подключается к агенту так же, как MCP-сервер с погодой – достаточно те же шаги.

Чтобы убедиться, что сама связка с геокодированием работает, давайте создадим агента, который будет запрашивать данные только из MCP геокодера.

Создание агента выглядит также, как мы это делали для агента с погодой – необходимо только ввести актуальные данные для агента с геокодером. Заготовку инструкции для агента, вы можете найти в приложенных материалах – “agent-instructions/only-geocoder-agent.txt”.

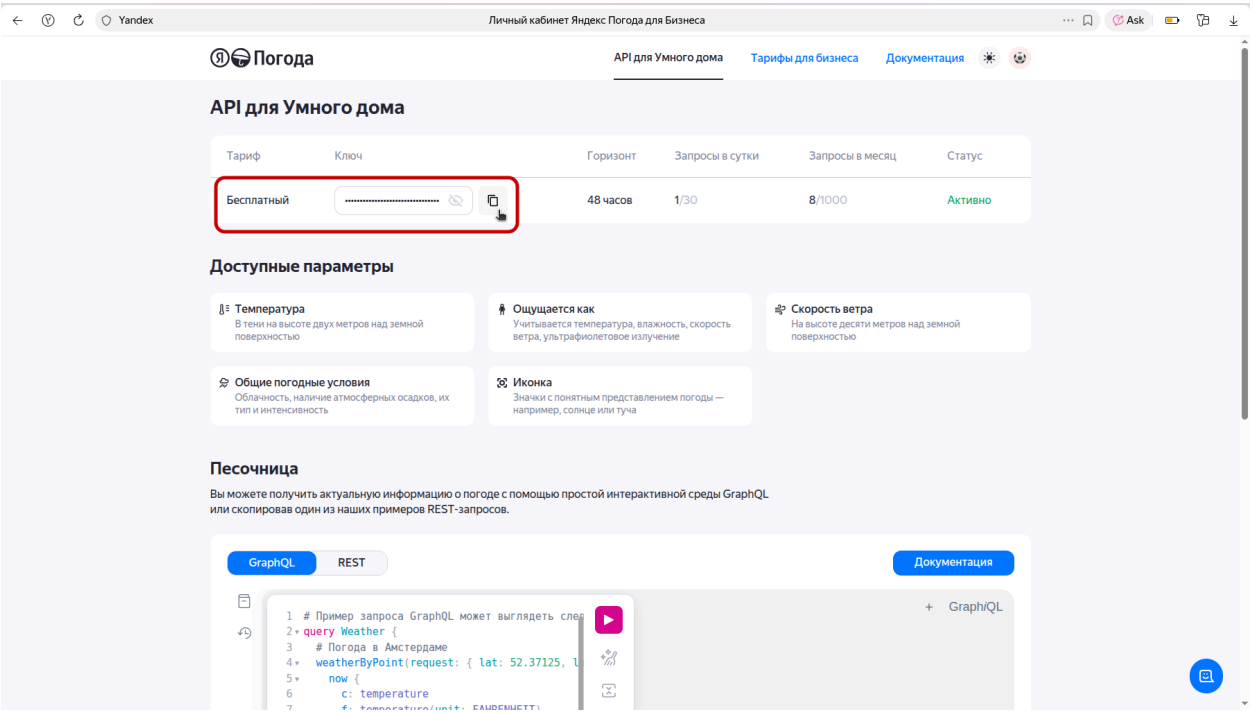
Таким образом, можно убедиться, что сам геокодер принимает название городов и других географических объектов передаёт агенту географические координаты этого объекта.

## Создание агента с геокодером и погодой

Агент создаётся аналогичным образом. Единственное, необходимо подключить сразу 2 MCP-сервера: сервер погоды и сервер геокодера. В остальном, механизм аналогичен тому, что мы с вами делали ранее

# How to get a Yandex Weather API key

- Go to the Yandex.Weather for Business website (only in Russian)
- Click the "Получить доступ" button and log in
- Copy the key (see screenshot below)



# Yandex Weather API | Useful documentation links

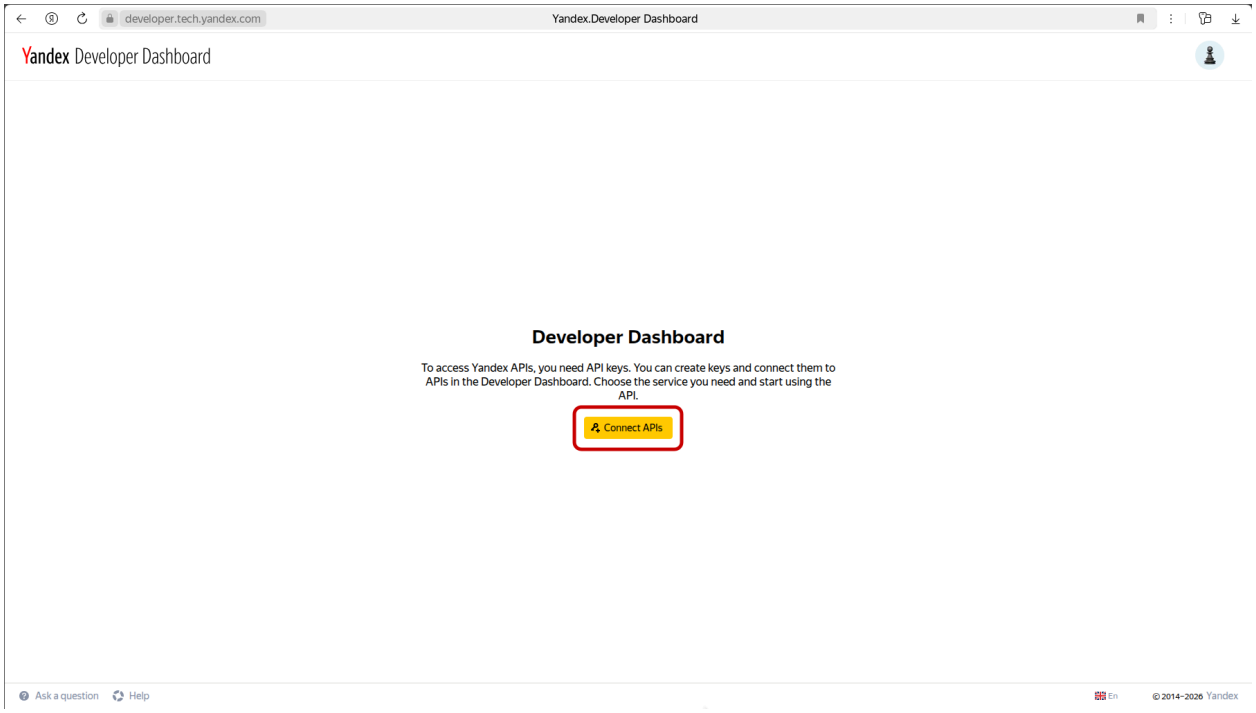
- Quick start
- Authorization

- Actual weather and current weather data
- Forecast
- Full GraphQL specification

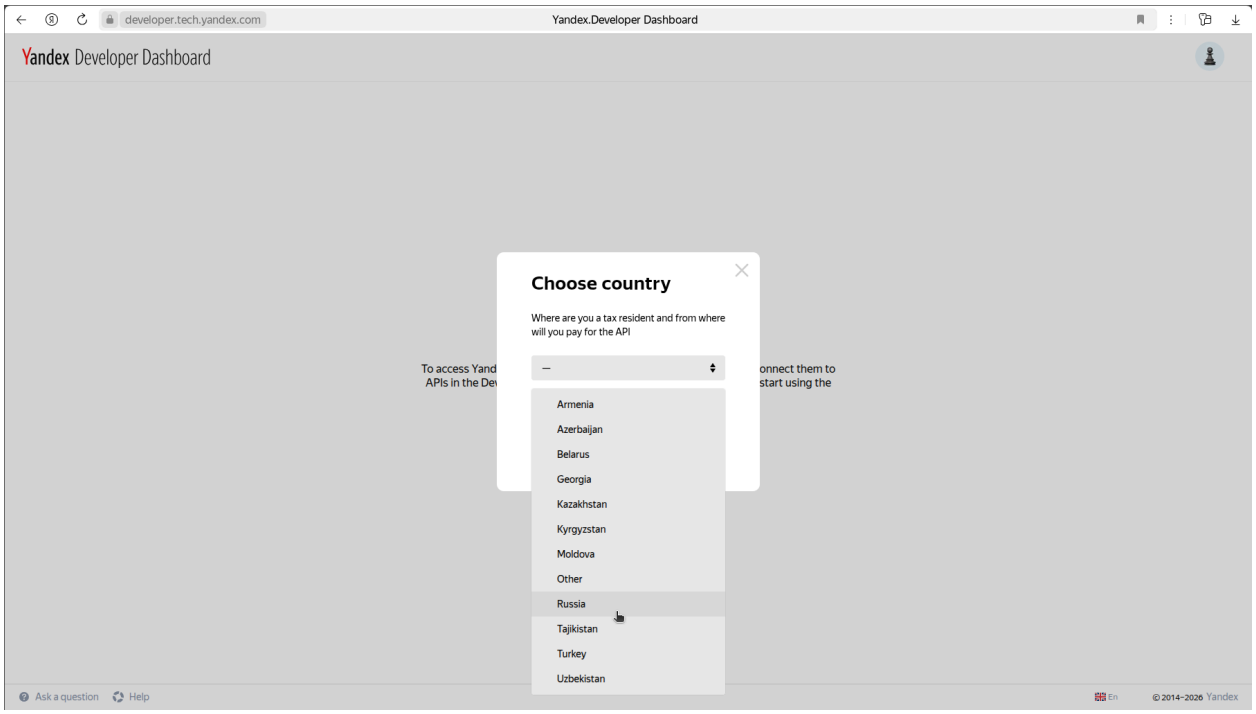
# Как получить ключ для API Яндекс Геокодер?

- Зайдите на сайт Yandex Geocoder
- Нажмите кнопку “Account” и авторизоваться

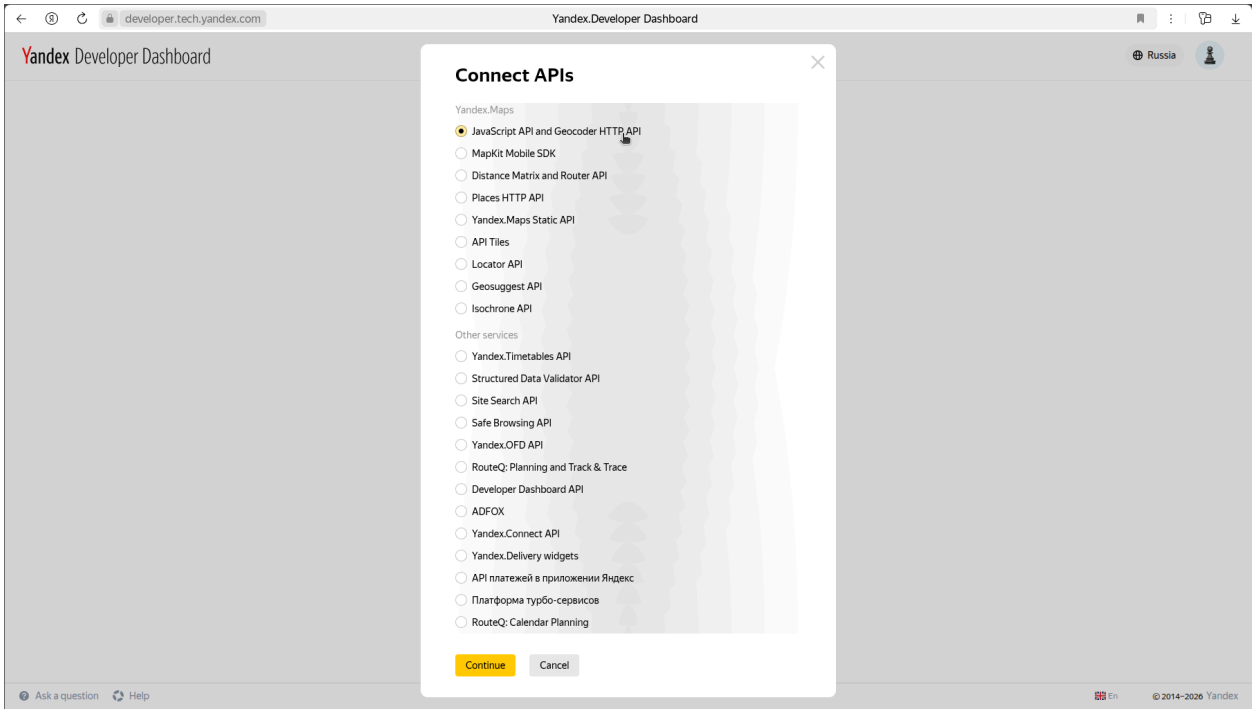
В Yandex Developer Dashboard нажмите кнопку “Connect APIs”



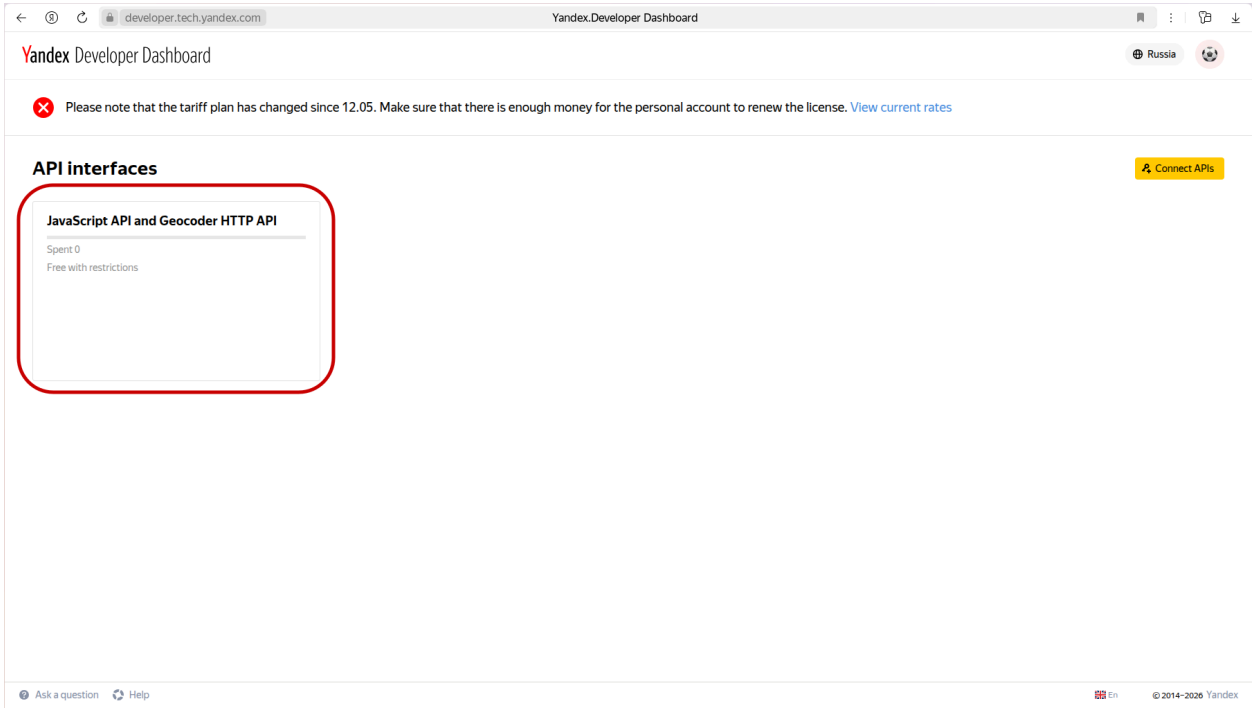

Выберете tax resident, который вам подходит



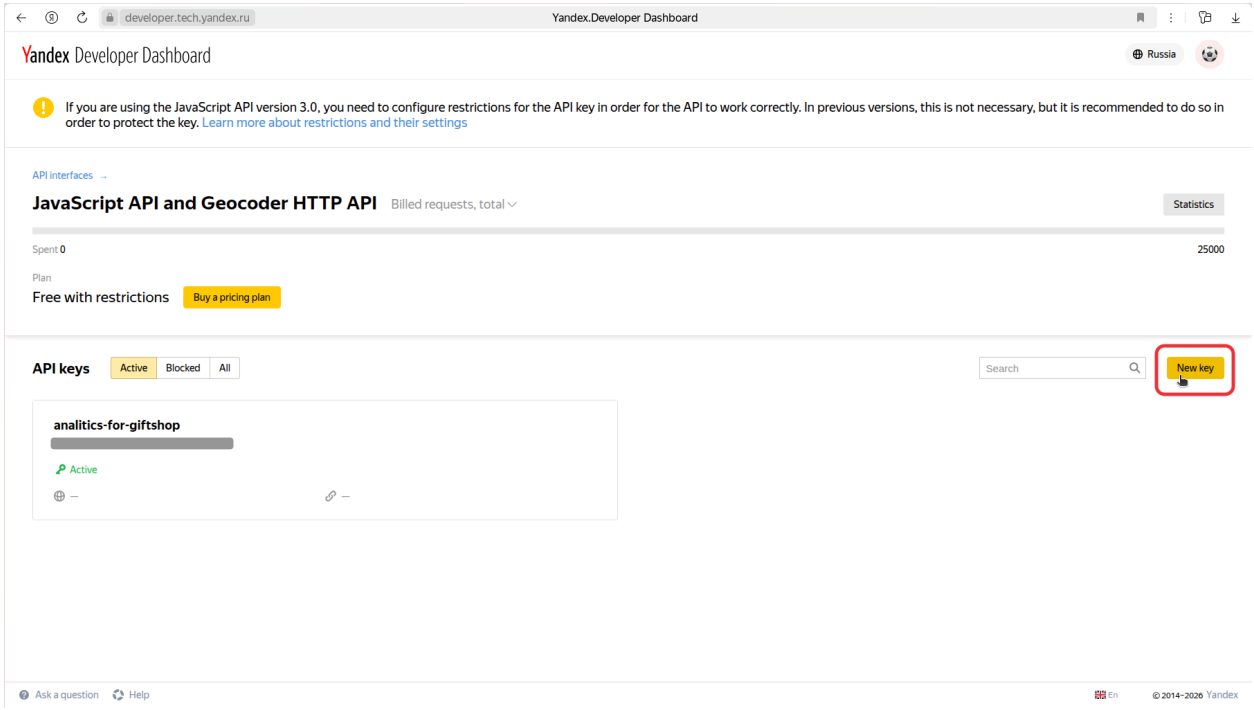

Выберете пункт “JavaScript API and Geocoder HTTP API” и нажмите “Continue”



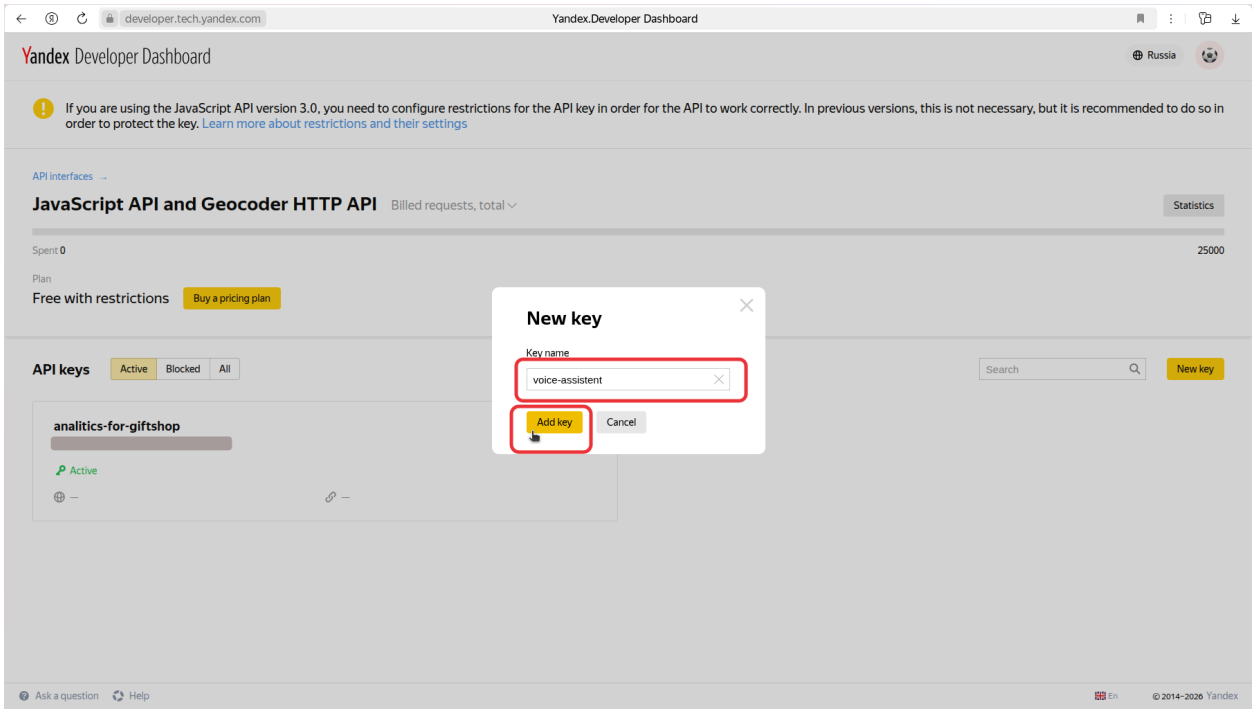

Если всё сделано правильно, вы увидите подобное окно. Нажмите на “JavaScript API and Geocoder HTTP API”



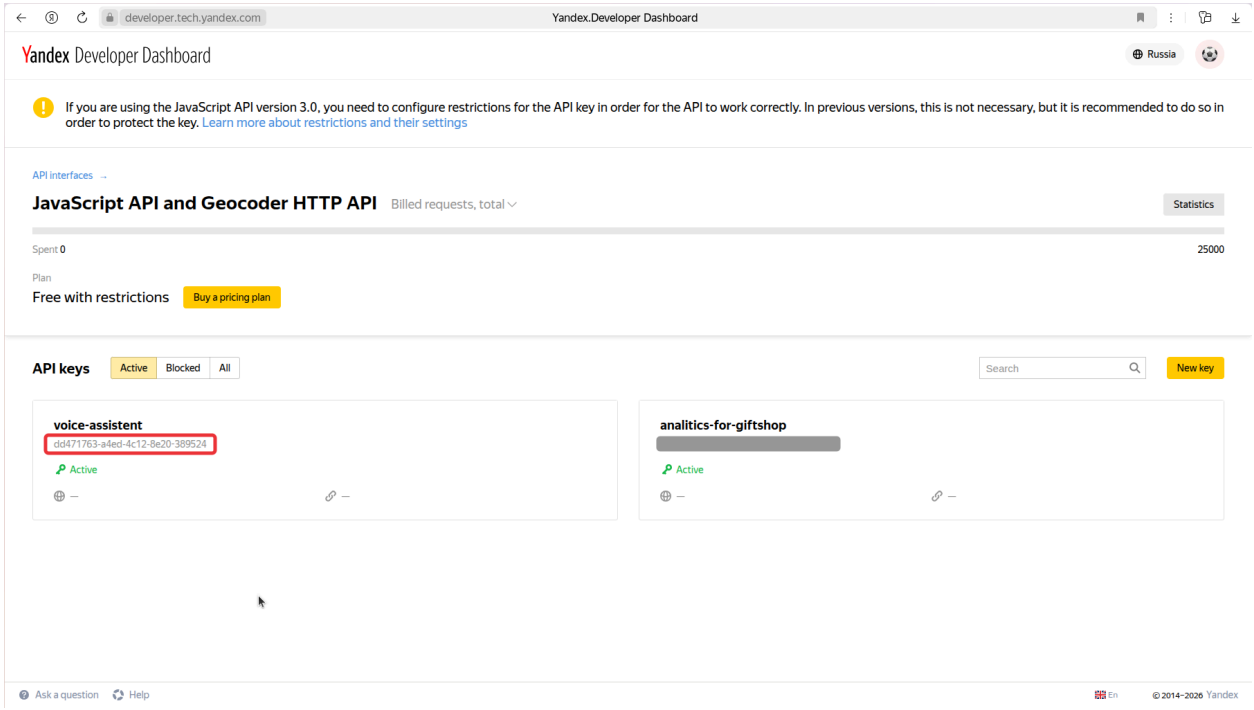

В открывшемся окне нажмите кнопку “New key”




Заполните поле “Key name” (например, “voice-assistent”) и нажмите кнопку “Add key”



Скопируйте ключ – см. изображение ниже




# Yandex Geocoder API | Useful documentation links

- Quick start
- Request format
- Response format
- Examples